

Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Desenvolvimento de Aplicações Distribuídas

Framework SPA

Component Framework, Vue.js

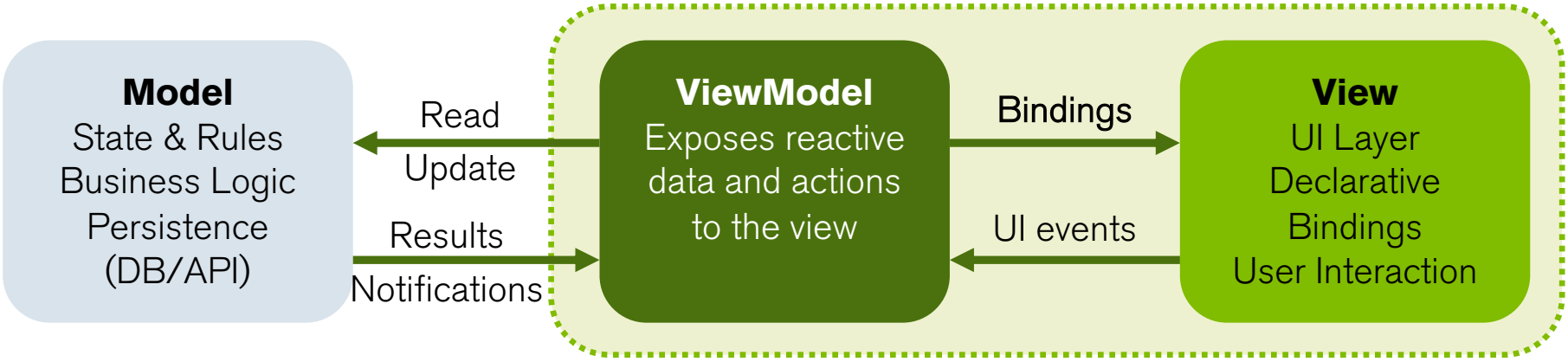
Marco Monteiro and Ricardo Gomes

1. Bootstrap & Basic Concepts
2. Vite & Development Tools
3. Templates
4. Reactivity
5. Components
6. Routing with Vue Router
7. State & Pinia
8. Composables

Bootstrap & Basic Concepts

Vue.js Overview

- Progressive SPA framework; approachable, incrementally adoptable.
- Component-based UI with Single File Components (SFCs).
- Composition API reactivity; declarative rendering updates the DOM.
- Follows the Model–View–ViewModel (MVVM) design pattern.
  - MVVM separates the UI (View) from application state (Model) via a ViewModel that exposes reactive data and actions to the View through declarative bindings.



Vue.js focuses primarily on the ViewModel and View layers

Kickoff: Install and run a Vue.js application

- Check documentation: https://vuejs.org/guide/quick-start.html
- Requires node.js (check version)

1. Create (scaffold) the project's folder:

```
npm create vue@latest
```

Then, on the project's folder:

2. Install external packages:

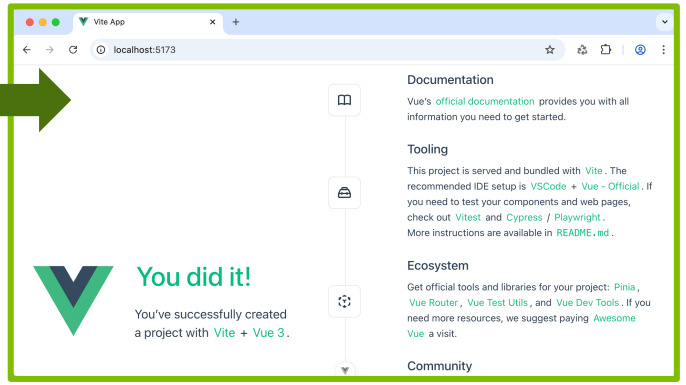
```
npm install
```

3. Run the project (using Vite):

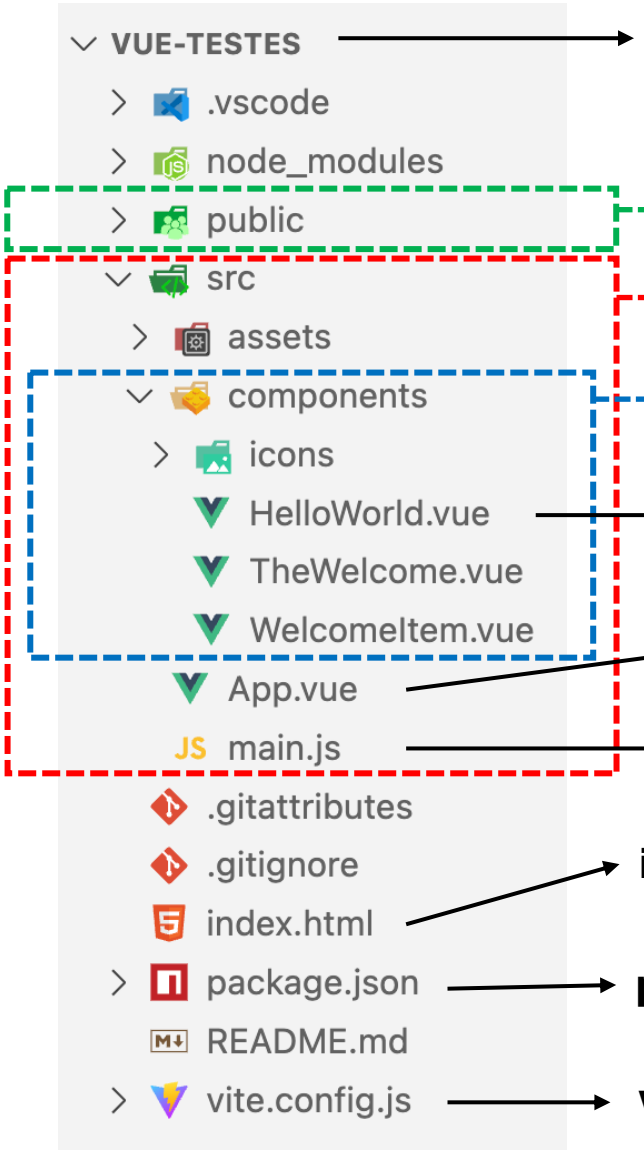
```
npm run dev
```

```
VITE v7.1.9  ready in 658 ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
➜  Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
➜  Vue DevTools: Press Option(⌥)+Shift(⇧)+D in App to toggle the Vue DevTools
➜  press h + enter to show help
```



Project Structure



The project folder

public folder – web server public folder

src folder – all the source code of the project

components folder – the components of the project

Example of a component

App.vue – Root component of the application

main.js – Project JavaScript entry point

index.html – HTML page where the Vue Application is mounted

package.json – npm project manifest

vite.config.js – Vite configuration (build/dev)

Project Configuration

- **package.json** – npm package manifest – declares dependencies & scripts; hosts tool configs.

Project metadata

npm scripts

Runtime Dependencies

Development Dependencies

```
{
  "name": "my-app",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "engines": {
  "node": "^20.19.0...
},
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "lint": "eslint . --fix",
  "format": "prettier ..."
},
  "dependencies": {
     "vue": "^3.5.22"
},
"devDependencies": {
  "@eslint/js": "^9.33.0",
  . . .
```

- **Vite.config.js** – configures Vite dev server, plugins, aliases, build.

```
import { fileURLToPath, URL }
  from 'node:url'

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import vueDevTools from 'vite-plugin-vue-
devtools'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    vue(),
    vueDevTools(),
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(
    new URL('./src', import.meta.url))
    },
  },
})
```

Entry Points: HTML

- **index.html** - where the Vue Application is mounted

```
<!DOCTYPE html>
<html lang="">
  <head>
    <meta charset="UTF-8">
    <link rel="icon" href="/favicon.ico">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>Vite App</title>
  </head>
  <body>
    <div id="app"></div>
      <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

Application will be mounted on this div (**#app**)

Uses ES6 modules

Project's JavaScript entry point

Entry Points: JavaScript

- **main.js** – project's entry point

Vite handles CSS processing to generate the final CSS output

In this example, it processes the plain main.css file during build

```
import './assets/main.css'

import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

function **createApp** is imported from vue core module

Application instance will be mounted on element (**#app**)

createApp function creates and returns an application instance

App is the root component of the application

Entry Points: Root Component

- **App.vue** – the root component
  - This component will be mounted on the selected HTML element (#app)
  - Component defined as a **Single-File Component (SFC)**
    - Code (model), Template (view) and styling on a single file

Code ***(Model)***

```
<script setup>
import HelloWorld from './components/HelloWorld.vue'
...JavaScript Code
</script>
```

<script setup> code block for SFC components

HelloWord – imported component

Template ***(View)***

```
<template>
   ...HTML
   <HelloWorld msg="You did it!">
   ....HTML
</template>
```

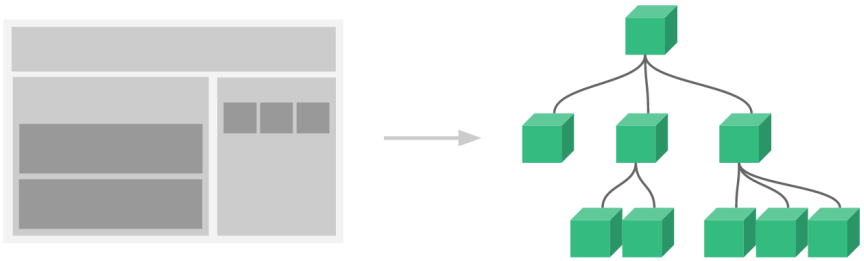
HelloWorld component is used here

Styles

```
<style scoped>
   ...CSS
</style>
```

Components - Concepts

- Reusable, isolated UI units
- Encapsulate template, logic, styles (on a single file when using SFC)
- Application UI is a component tree with a single root and descendants
- Composition via parent–child relationships (parent nests children as custom elements)



- Like native HTML tags (that include attributes & events), components also expose props and emit events.
- Example of using a component (registered as `UserCard`) inside a parent template.

```
<UserCard name="Ada" @select="onSelect"/>
```

prop event

Component Basics & Data Binding

- **Counter.vue** – example of a Single File Component (SFC)
- Declarative rendering: template declaratively defines the View; Vue updates the DOM.
- Data binding: model state changes automatically updates the View.
- Events: View interactions trigger model logic via declared handlers.

```
<script setup>
import { ref } from 'vue'
const counter = ref(0)

const increase = () => {
   counter.value++
}
const decrease = () => {
   counter.value--
}
</script>
<template>
   <div>
      <p>Counter: {{ counter }}</p>
      <button v-on:click="increase">+</button>
      <button @click="decrease">-</button>
   </div>
</template>
```

Data binding

Events

- To use previous component, we must import and register it:

```
<script setup>
  import Counter from './components/Counter.vue'
  ...
<template>
   <Counter />
```

Vite & Development Tools

Vite in Vue.js projects

- Modern dev server and build tool.
  - **npm run dev** — start dev server with HMR.
  - **npm run build** — create optimized production bundle.
  - **npm run preview** — serve built app locally.
- Development
  - Native ECMAScript modules (ESM).
  - Instant Hot Module Replacement (HMR).
  - Path aliases simplify imports (@ → /src).
  - Loads .env files; exposes VITE_* vars.
- Build (for production)
  - Rollup bundles: optimizes and minimizes modules.
  - Code splitting: on-demand loading.
  - Tree shaking & minification remove unused code.
  - Hashing for cache busting.
  - CSS extracted and optimized.
- Vue integration
  - .vue files compiled by @vitejs/plugin-vue.
  - Support <script setup> and scoped CSS.
  - HMR updates individual components instantly.
- Assets management
  - Imports (CSS, images, fonts) become URLs.
  - Files in `public` folder copied as-is.
  - Optimized and hashed in production.
- Dev tooling
  - Dev server proxy for backend APIs.
  - Works with ESLint, PostCSS, TypeScript.
  - Minimal configuration required.

Development workflow

- Start the development server:

```
npm run dev
```

- Development server runs on a specific port.
  - No need to start a backend web server
  - Default port is **5173** (configurable)

```
VITE v7.1.9  ready in 658 ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
```

- Typical code when running in development mode
  - Uses ECMAScript Modules (ESM)
  - CSS served by the development server at: `http://localhost:<PORT>`

```
<link rel="stylesheet" href="http://localhost:5173/@id/virtual:...">

<img ... src="/src/assets/logo.jpg" ... >

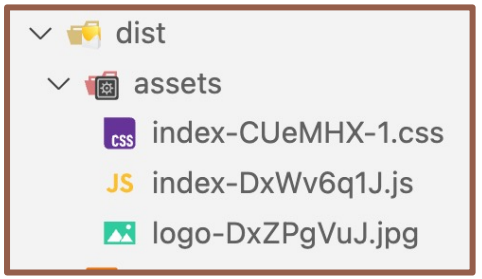
<script type="module" src="/src/main.js"></script>
```

Build for production

- Builds optimized bundles
  - Transpiles JS and CSS into production-ready files.
  - Copies bundles and assets to the **`dist`** folder.
- **`dist/`** folder ready for deployment
  - Copy its contents to the production web server.
  - Test locally with `npm run preview` (default port 4173).
- Typical code after build:
  - Uses bundled JS/CSS instead of separate ESM modules.
  - Asset filenames are hashed for cache busting

```
npm run build
```

```
vite v7.1.9 building for production...
✓ 24 modules transformed.
dist/index.html                  0.47 kB
dist/assets/logo-DxZPgVuJ.jpg    7.55 kB
dist/assets/index-CUeMHX-1.css   3.71 kB
dist/assets/index-DxWv6q1J.js   70.74 kB
```



Note: JS, CSS, and image filenames match those generated by **`npm run build`**

```
<link rel="stylesheet" ... href="/assets/index-CUeMHX-1.css">

<script type="module" ... href="/assets/index-DxWv6q1J.js">

<img ... src="/assets/logo-DxZPgVuJ.jpg" ...>
```

Assets handling with Vite

- Imports of images, fonts, or media are converted into URLs
- During development, assets are served directly by the dev server
- During build, assets are optimized, hashed, and copied to /dist/assets/
- Files placed in /public/ are copied as-is and available at /
- Example (HTML):

```
<template>
  <img src="/src/assets/logo.jpg" .../>
</template>
```

- Example (JS Module):

```
<script setup>
import logo from '@/assets/logo.jpg'
</script>

<template>
    <img :src="logo" .../>
</template>
```

- Generated source code (examples)

Development Server:

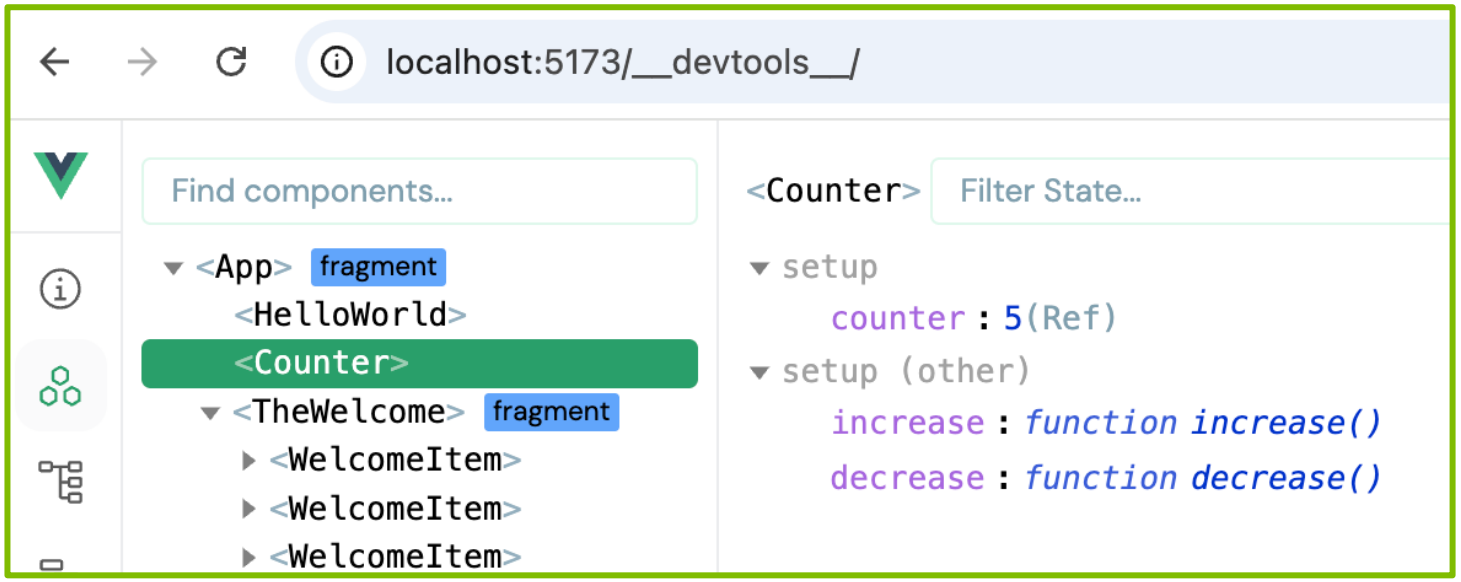
```
<img ... src="/src/assets/logo.jpg" ... >
```

Production Build:

```
<img ... src="/assets/logo-DxZPgVuJ.jpg" ...>
```

Development tools

- **Vue Devtools** — inspect component tree, props, reactive state, etc.
  - During development: `http://localhost:<PORT>/__devtools__/`



- VS Code Extensions:
  - **Vue (Official)** – formerly Volar; provides Vue 3 language features and type checking
  - ESLint + Prettier — code style and consistency.

Templates

Vue.js Templates

- Declarative templates: no imperative DOM manipulation
- Template syntax: interpolation, attribute and event bindings, directives, modifiers
- Data binding keeps Model and View in sync
  - Model changes automatically update the View
  - View interactions emit events to update the Model
- Reactivity updates the View automatically — no manual DOM calls
- Vue updates the DOM using a virtual DOM
  - Virtual DOM: in-memory tree; Vue computes diffs and updates only the changed parts of the browser DOM

Template Syntax Overview

- **Interpolation**: `{{ expression }}` renders text (no attributes)
- **Attribute binding**: `v-bind` / `:` for dynamic attributes/props values
- **Event binding**: `v-on` / `@` for event handlers
- **Directives**: `v-if`, `v-else`, `v-else-if`, `v-show`, `v-for`, `v-model`, …
  - Special **`v-*`** attributes that controls rendering and behavior
- **Modifiers**: `.prevent`, `.stop`, key (e.g., `.enter`), form (e.g., `.trim`)

Bold text indicates JavaScript expressions

Interpolation – `{{ … }}`

Attribute binding - `:src`

Event binding - `@click`

Directive - `v-if`

Directive - `v-model`
Modifier - `.trim`

```
<p>{{ person.email.toLowerCase() }}</p>

<img :src="person.avatarUrl" .../>

<button @click="eventHandler">Save</button>

<div v-if="person.isStudent">Student</div>

<input v-model.trim="person.address"/>
```

Interpolation and Bindings

- Attribute and event binding **shorthands**:

```
<img     v-bind:src="logo" .../>
```
==
```
<img     :src="logo" .../>
```

```
<button v-on:click="eventHandler" ...>
```
==
```
<button @click="eventHandler" ...>
```

- Interpolation is text-only; use `v-bind` / `:` for attributes/props:

```
<img :src="logo" .../>
```
✓

✗
```
<img :src="{{ logo }}" .../>
```

- Values are JavaScript expressions – not plain text

```
<img :src="logo" .../>
```
```
<img :src="'http://a.com'" .../>
```
```
<p> {{ personName }}</p>
```
✓

These are NOT valid JavaScript expressions:

✗
```
<img :src="http://a.com" .../>
```
```
<p> {{ texto simples }}</p>
```

- Event handlers can be inline expressions or **function references**

```
<button @click="calc() + 1" ...>
```
```
<button @click="functionName" ...>
```

Conditionals & Loops Directives

- **v-if**, **v-else**, **v-else-if**, create or destroy UI elements on a condition
- **v-show**, show or hides (toggles) UI elements
  - UI elements are always created
  - Better for frequent toggles
  - Does not support v-else or v-else-if

```
<div v-show="isStudent">
   Student
</div>
```

```
<div v-if="isStudent">
   Student
</div>
```

```
<div v-if="type==='S'">
   Student
</div>
<div v-else-if="type==='T'">
   Teacher
</div>
<div v-else>
   Other user
</div>
```

- **v-for**, loops (repetitions). Render collections (usually arrays)

```
<div v-for="item in items">
   {{ item }}
</div>
```

```
<div v-for="obj in objets" :key="obj.id">
   {{ obj.name }}
</div>
```

If possible, always provide a **key**

Just loop (no collection):

```
<div v-for="n in 10">
  Number {{ n}}
</div>
```

Don't combine v-if and v-for on the same element.

```
<div v-for="obj in objets" v-if="obj.ok">
```

Class & Style Bindings

- Bindings with `:class` and `:style` expect string, array, or object
- Vue merges static and dynamic classes or styles
- Object dynamic binding toggles by truthiness:
  - Object property names map to class names; applies when their values are truthy

Static `class` ("bg-light") – always applied

Dynamic `:class` - depends on the expression

```
<div class="bg-light" :class="{active: isActive, 'text-danger': isError}">
```

Class name : Boolean expression

➢ If `isActive` is true and `isError` is false, the result class attribute value would be: **`class="bg-light active"`**

- Array dynamic binding composes a set of classes from the array
  - Elements of the array will be applied as class names
  - Elements can be strings (class names) or objects with the same syntax as object bindings

```
<div class="bg-light" :class="['isActive ? 'active': '', strValue, 'clsA'] ">
```

```
<div :class="[{active: isActive, 'text-danger': isError}, 'clsA']">
```

```
<div class="bg-light" :class="arrayValue">
```

Forms & Two-Way Binding

- **`v-model`** creates two-way binding and keeps input and state in sync

```
<input v-model="name">
```

Input field will show the value of **`name`** (one-way binding)

Changes on the input will change the value of **`name`**

- Syntactic sugar for `:value` and `@input`

  Under the hood, Vue transforms the previous example into:

```
<input :value="name" @input="e => name = e.target.value ">
```

- Works with native form inputs; custom components can implement `v-model` too.

- **`v-model`** supports these modifiers: `.trim`, `.number`, `.lazy`

```
<input v-model.trim="name">
```
Removes trailing spaces

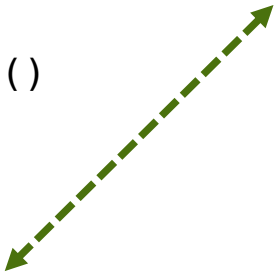
```
<input v-model.number="age">
```
Automatically typecast as a number

```
<input v-model.lazy="email">
```
Applies the change when you finish editing, not on each keystroke

Template Refs

- The **`ref`** attribute lets Vue access the underlying DOM element or the instance of a child component.

```
<script setup>
import { useTemplateRef, onMounted } from 'vue'
const input = useTemplateRef('my-input')
onMounted(() => {
   input.value.focus()
})
</script>
<template>
   <input ref="my-input" />
</template>
```



- Best practice in Vue: avoid direct DOM access—use Vue refs/bindings. Only use direct DOM for third-party integrations and run that after onMounted.
  Avoid `document.*` - e.g., `document.getElementById('abc')`

List of Directives & Modifiers

- Directives

  It is possible to add custom directives

`v-bind :` `v-on @`

`v-if` `v-else-if` `v-else` `v-show`

`v-for` `v-model` `v-slot`

`v-text` `v-html`* (trusted only)

`v-pre` `v-once` `v-memo` `v-cloak` `v-is`

* Renders HTML directly. Only for trusted content and never on user-provided content

- Modifiers

  Summary of modifiers

`.stop .prevent .capture .self .once .passive`

`.left .right .middle`

`.enter .tab .esc .space .up .down .left .right .ctrl .alt .shift ..meta .exact`

`.trim .number .lazy`

- Modifiers can be combined. Example:

```
<div @keydown.alt.enter="eventHandler">
```

Reactivity

Refs & Reactivity Basics

- Reactive state is data that Vue tracks and binds to the template
  - When it changes, Vue re-renders the affected parts of the view (kept in sync automatically)
  - Vue updates the real DOM by diffing a Virtual DOM and patching only what changed

```
<script setup>
import { ref } from 'vue'
const counter = ref(0)
const increase = () => {
   counter.value++
}
</script>
<template>
  <p>{{ counter }}</p>
  <button @click="increase">
 + </button>
</template>
```

In templates, refs read like plain values; in JavaScript use `.value`

- `counter` is not a number; it's a `ref` object whose value lives in `counter.value`
- Conceptually, a ref object exposes a reactive getter/setter; Vue tracks reads in the getter and triggers updates on writes in the setter.

```
const counter = {
  _value: 0,
  get value() {
    track(this)
    return this._value
  },
  set value(v) {
    this._value = v
    trigger(this)
  }
}
```

Conceptual pseudo-code

Under the hood:

- getter **tracks** access
- setter detects changes and **triggers** updates

ref() vs reactive()

- `ref()` - wraps the inner value in a special object
- `reactive()` - another way to declare reactive state (only object types). Makes an object itself reactive – returns a proxy of the original object (≠ original object)

```
<script setup>
import { ref, reactive } from 'vue'
const counter = ref(0)
const state = reactive({ count: 0 })
const inc = () => {
  counter.value++
  state.count++
}
</script>
<template>
 <button @click="increase">
 + </button>
 <p>ref: {{counter }}</p>
 <p>reactive: {{ state.count }}</p>
</template>
```

- **`ref(x)`** – wrapper with `.value`
- `.value` is a direct reference to the data
- You can replace the whole value
- Auto-unwrap in templates `{{ count }}`

- **`reactive(obj)`** – proxy of the object.
- Only objects types (objects, arrays, Map, Set)
- The proxy is not the original object
- You can mutate the properties, but not the object (proxy)
- Destructuring loses reactivity
- Read normally in templates `{{ state.count }}`

Computed

- A `computed` is a `ref` whose value is derived by a function

```
<script setup>
  import { ref, computed } from 'vue'
  const imgIdx = ref(0)
  const imgSrc = computed(() =>
 `https://picsum.photos/id/${imgIdx.value}/600/400`)
  const imgNext = () => imgIdx.value++
</script>

<template>
    <button @click="imgNext">+</button>
    <img :src="imgSrc" alt="Random Image" />
</template>
```

- `computed` is read-only by default, but you can make it writable with `{ get, set }`
- `computed` values are cached
  - On first access, Vue runs the getter, tracks its reactive dependencies, and caches the result.
  - When any dependency changes, the computed is marked dirty and recomputed on the next read; otherwise, the cached value is returned.

```
const imgSrcFunction = () =>
`https://picsum.photos/id/${imgIdx.value}/600/400`
. . .
<img :src="imgSrcFunction()" />
```

Alternative that uses a function, will work, but the function will be executed every time any reactive data changes on the component

A method call in the template runs on every render (no caching),
A computed value is cached and recomputes only when its dependencies change.

Watch

- `watch` runs code when reactive data changes

When `counter` changes, watch callback is called (example works for refs or computed):

```
import { ref, watch } from 'vue'
  const counter = ref(0)
  watch(counter, (newVal, oldVal) => {
     console.log(`Counter changed from ${oldVal} to ${newVal}`)
  })
```

Watch a **getter** (runs when the getter's returned value changes):

```
const x = ref(0)
const y = ref(0)
watch(
  () => x.value + y.value,
  (sum) => {
    console.log(
`x+y changed to ${sum} `)
  }
)
```

To watch a reactive property, use a getter:

```
const state = reactive({ count: 0 })
watch(
  () => state.count,
  (count) => {
     console.log(`count has changed to ${count} `)
  }
)
```

Check https://vuejs.org/guide/essentials/watchers.html for more details

Components

Component Definition

- Vue provides two APIs to define components: Composition API and Options API.
  - Options API — legacy style from Vue 2 (still supported; not deprecated).
  - Composition API — introduced in Vue 3.
  - Both are supported in Vue 3; this course focuses on Composition API.
- Ways to define a component:
  1. Composition API with Single File Component (SFC) - <script setup>
     - A single .vue file with three sections: <template>, <script setup>, <style>
     - Full tooling: HMR, template type-checking, CSS scoping.
     - **Preferred approach** — we will use this predominantly in this course.
  2. Composition API (code-only)
     - A JavaScript module exporting a component with setup()
     - Useful for tiny, renderless, or dynamic components; clarifies how SFCs work under the hood.
  3. Options API (legacy style — Vue 2)
     - Component defined by an options object: props, data, methods, lifecycle, …

Component Definition

Options API:

```
export default {
  data() {
    return {
      counter: 0,
    }
  },
  methods: {
    increase() {
      this.counter++
    },
  },
}
```

- Rigid structure
- Object-based style
- Vue proxies **`this`**
- Harder to reuse logic across components

Composition API – setup()

```
<script>
import { ref } from 'vue'
export default {
  setup() {
    const counter = ref(0)
    const increase = () => {
      counter.value++
    }
  return {counter, increase}
  }
}
</script>
```

Exposed elements

- "Plain" JavaScript code
- No **`this`** in Composition API
- Flexible structure
- Easily extendable through composition
- setup() function returns an object with the elements exposed to the template

Composition API <script setup>

```
<script setup>
import { ref } from 'vue'

const counter = ref(0)

const increase = () => {
  counter.value++
}
</script>
```

- Syntax sugar over Composition API - setup()
- The SFC compiler transforms <script setup> into a standard component using setup()
- Top-level bindings are **automatically exposed** to the template

Component Registration

- Parent components nest children as custom elements.
- Component **registration** associates/maps a component to a custom element

- **Local registration** (on the parent component – using SFC):

```
<script>
import AboutPage from './AboutPage.vue'
</script>
<template>
   <AboutPage />
</template>
```

With SFC components, local components are automatically registered with the name used in the import statement

- **Global registration** (`main.js` file)– component available globally:

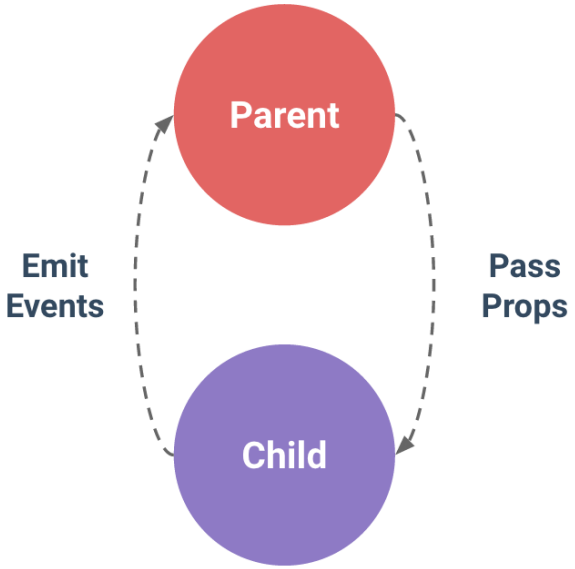
```
import AboutPage from './AboutPage.vue'
const app = createApp(AboutPage)
// ...
app.component('AboutPage', AboutPage)
```

Name of the custom element:

```
<AboutPage />
```

Custom element <AboutPage) can se used in any other component's template

Composing Components



- How do parent and child communicate?
  - Parent passes data down via props
  - Child notifies the parent by emitting events
  - One-way data flow: props down, events up

- Parent renders the child:

```
<UserCard name="Ada" @select="onSelect"/>
```

- The child is parent-agnostic:
  - It declares props and emits events as its public interface.
  - The parent supplies prop values and registers event handlers.

Defining Props

- Alternatives for a component to define a `prop`:

- Array syntax (name only):

```
<script setup>
const props = defineProps([
  'name',
  'age',
  ])
</script>
```

- Object syntax (with validation):

```
const props = defineProps({
  'name': String,
  'age': {
     type: Number,
     required: true,
     validator: (value) => value >= 0
   }
})
```

https://vuejs.org/guide/components/props.html#prop-validation

- Access prop value in component **code**:

```
<script setup>
   . . .
   console.log(props.name))
```

- Access prop value in **template**:

```
<template>
<p>Name: {{ name }}, Age: {{ age }}</p>
```

- Props are read-only inside the component where they are defined.
- Prop values are provided by the parent component (data flows down):

```
<UserCard name="John" :age="30"/>
```

Passing Prop Values from the Parent

- Parent template using a child component (`BlogPost` is the child)

```
<template>
  <BlogPost title="Static value – always a string"/>
  <BlogPost v-bind:title="post.title"/>
  <BlogPost :title="post.title + ' by ' + post.author.name"
            :likes="42"
            :comment-ids="[234, 266, 273]"
            :author="post.authorObject" />
</template>
```

- A prop behaves like a custom attribute defined by the child component
- Prop values can be static (e.g., `title`) or dynamic (e.g., `:title`)
- Static values are always strings
- Dynamic values are JavaScript expressions that return any type: string, number, boolean, array, object, etc.

Emitting Events

- Events (emits) let a child component notify its parents or send data back

```
<script setup>
  const emit = defineEmits([
     'selected',
  ])
  const select = () => {
     emit('selected', userName )
  }
```

Declare event name

Emit the event anywhere in your code

Attach custom data to the event

Examples of data passed to the event:

```
emit('selected')
emit('selected', userName)
emit('selected', v1, { name: username }, [a,b])
```

- Events are handled by the parent:

```
<template> <UserCard name="John" :age="30" @selected="handleSelected" />
```

```
<script setup>
  const handleSelected = (name) => { /* handle event here */ }
```

Props – One-Way Data Flow

- Props flow down - events go up — this is Vue's one-way data flow pattern
- When the parent updates a prop, the new value flows down to the child — never the other way around.
  - This prevents child components from accidentally mutating the parent's state
- Props are read-only inside the component where they were defined
  - To change a prop value, emit an event to notify the parent so it can update the state.

```
<script setup>
  const props = defineProps(['initialCounter'])

  props.initialCounter = 5
```



props are read-only

- Object or array props are read-only, but their nested properties are still mutable!

```
<script setup>
  const props = defineProps(['obj', 'arr'])

  props.obj = { name: 'John Doe' }
```



```
// not valid – props are read-only

// object or array nested properties are not readonly!
  props.obj.name = 'Jane Doe'
  props.arr[2] = 'new value for the third element'
```

v-model – Two-Way Binding

- Custom components can support `v-model` directive for two-way bindings

```
<MyComponent v-model="counter" />
```

- Under the hood, the Vue.js compiler expands previous code to:

```
<MyComponent :modelValue="counter"
             @update:modelValue="newValue => counter = newValue"/>
```

- For a component to support `v-model` it must include the prop named `modelValue` and emit the event `update:modelValue`

```
const props = defineProps(['modelValue'])
const emit = defineEmits(['update:modelValue'])
const increase = () => {
  emit('update:modelValue', props.modelValue + 1)
}
```

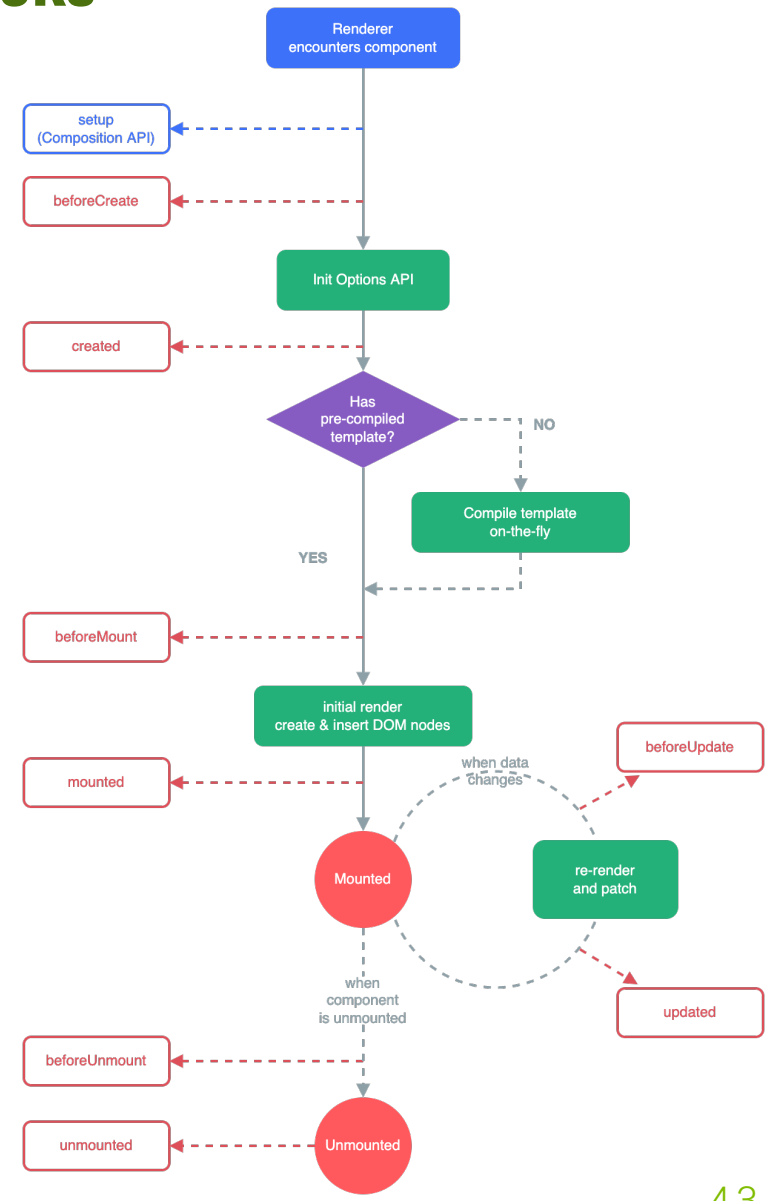
- Starting in Vue 3.4, the **recommended approach** is using the `defineModel()` macro:

```
const model = defineModel()
const increase = () => {
  model.value++
}
```

```
<template>
  value is {{ model }}
```

Lifecycle & Lifecycle Hooks

- The component **lifecycle** represents the stages a Vue component goes through (from creation to destruction)
- Lifecycle **hooks** are functions that let you run code at specific points in that lifecycle.
  - The diagram shows when each hook (e.g., `onMounted`, `onUpdated`, `onUnmounted`) is triggered.



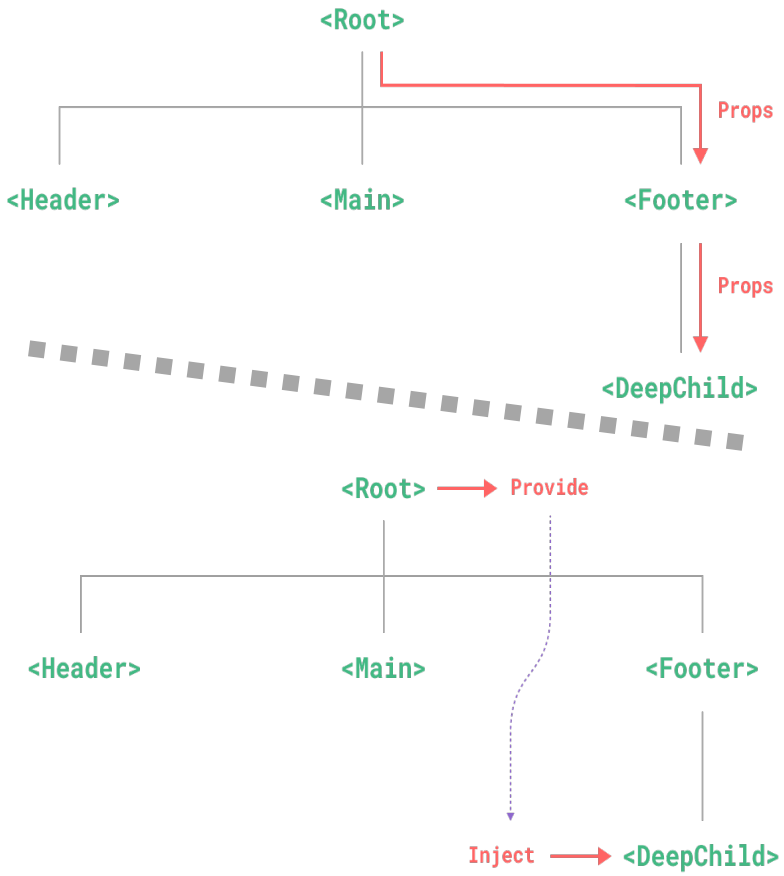
- **`onMounted()`**- runs after the component is inserted into the DOM. Commonly used to fetch data, interact with the DOM, or initialize libraries that require the element to exist.

```
import { onMounted } from 'vue'
onMounted(() => {
   // initialize
})
```

- `onUpdated()` — runs after reactive data updates re-render the DOM.
- `onUnmounted()` — cleanup logic when the component is removed (e.g., event listeners, timers)

Provide / Inject

- Enables passing data from an ancestor component to any descendant without prop drilling (skipping intermediate components).
- Useful for global-like context, such as configuration, theme, or shared services.
- `provide()` makes data available to all descendants.




```
// main.js file:
import { io } from 'socket.io-client'
const app = createApp(App)
const socket = io('http://localhost:3000')
app.provide('socket', socket)
```

- `inject()` allows a child (or deep child) to access that provided data.

```
import { inject } from 'vue'
const socket = inject('socket')
// socket reference is the same as provided
socket.emit('echo', 'message to server')
```

Other Component-Related Features

(*Explore these in Vue.js documentation for more details*)

- **`Slots`** – pass content from parent into a component

  ```
  <MyCard> <p>Custom content</p> </MyCard>
  ```

- **`Async Components`** – dynamically load components on demand
- **`Dynamic Components`** – switch component at runtime using

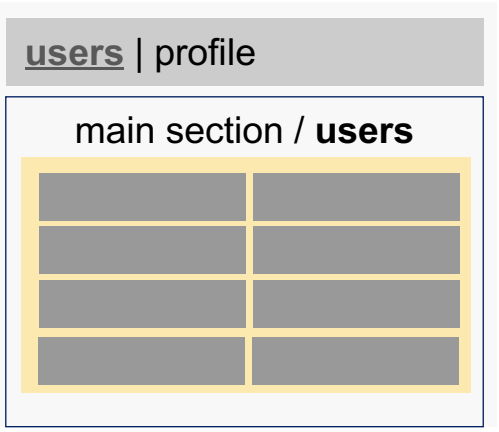
  ```
  <component :is="...">
  ```

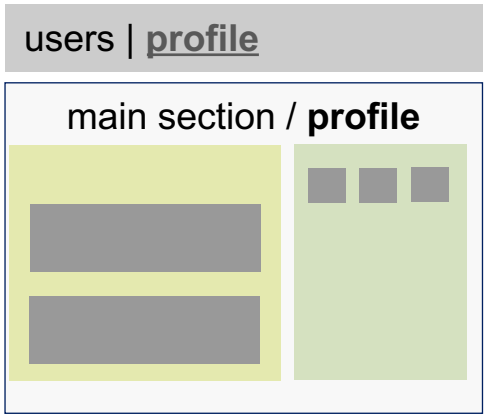
- **`KeepAlive`** – cache inactive component instances; preserve state
- **`Teleport`** – render in a different DOM location (e.g., modals)
- **`Transitions`** – apply enter/leave animations and visual effects
- **`Custom Directives`** – define reusable DOM behavior beyond built-ins

Routing with Vue Router

Vue Router

- A SPA runs on a single page, but its content changes through internal navigation handled by the client-side router
- Vue Router (https://router.vuejs.org/) is the official routing library for Vue.js.
- Maps URL paths to components






```
routes: [
  {
    path: '/',
    name: 'home',
    component: HomePage,
  },
  {
    path: '/users',
    name: 'users',
    component: Users,
    children: [
      {
        path: '/:id',
        name: 'user-detail',
        component: User,
      },
      {
      path: '/profile',
      name: 'user-profile',
      component: UserProfile,
      },
    ]
  }
]
```

Templates: <router-link> and <router-view>

- **<RouterLink>** creates navigation links
  - Supports named routes and dynamic params

```
<RouterLink to="/users">Users</RouterLink>
<RouterLink :to="{ name: 'user-profile'}">Profile</RouterLink>
<RouterLink :to="{ name: 'user-detail' , params: { id: 3 }}">Details</RouterLink>
```

- **<RouterView>** renders the matched route component:
  - Wrap with `<KeepAlive>` to cache inactive views
  - Wrap with `<Transition>` for animated view changes

```
<RouterView />
```

```
<RouterView v-slot="{ Component }">
  <KeepAlive>
    <component :is="Component" />
  </KeepAlive>
</RouterView>
```

```
<RouterView v-slot="{ Component }">
  <Transition name="fade">
    <component :is="Component" />
  </Transition>
</RouterView >
```

```
<RouterView v-slot="{ Component }">
  <Transition name="fade">
    <KeepAlive>
      <component :is="Component" />
    </KeepAlive>
  </Transition>
</RouterView>
```

Definition and History Mode

- `createRouter({ history, routes })` bootstraps navigation

```
import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
   history: createWebHistory(import.meta.env.BASE_URL),
   routes: [ /* Route Mapping */ ],
})
```

- History mode defines how the router manages URLs and browser history behavior

`http://site.com/users` **Web History**: clean URLs; server configuration required*

`http://site.com/#/users` **Hash History**: # URLs; works on static hosting

Memory History: SSR/tests only; no address bar updates

* Production requires a server rewrite to index.html for all non-asset routes. Vite handles this automatically in development. See “Example Server Configurations”: https://router.vuejs.org/guide/essentials/history-mode.html#example-server-configurations

Route Parameters & Prop Mapping

- Route **path** can include parameters:

```
{ path: '/users/:id', component: A}
```

Access the value of `:id` parameter in the component

```
import { useRoute } from 'vue-router'
const route = useRoute()
const valueOfParam = route.params.id
```

- Passing **props** to the component:

```
// Boolean mode: parameter values are passed as props with the
//               same name (always as string values)
   { path: '/users/:id/:role', component: B, props: true},

// Object mode: object will be set as the component props as-is
   { path: '/users', component: C, props: { showEmail: false } },

// Function mode: function that returns an object with the props
   { path: '/users/:id', component: D, props: route => ({
        textToSearch: route.query.q,
        id: Number(route.params.id)
   })
```

Programmatic Navigation and Navigation Guards

- Vue Router provides access to the router and the current route instances

```
import { useRoute, useRouter } from 'vue-router'
const route  = useRoute()     // Current route
const router = useRouter()    // Router instance
```

- Router instance programmatic navigation:

```
router.push({ name: 'users', params: { id: 13 } }) // Navigate to named route
```

```
router.back()
router.forward()
router.go(-1) // Same as back()
```

```
// Replace current route
// (avoid stacking history)
router.replace({ name: 'home' })
```

- Navigation guards:
  - Primarily used to guard navigations either by redirecting it or canceling it.
  - Ways to hook into the route navigation process: globally, per-route, or in-component.

```
router.beforeEach((to, from) => {
  // return false: cancel the navigation
  // (canAccess must be implemented)
  if (!canAccess(to.name)) {
    return false
  }
})
```

Handling Route Param Changes

- When the route param value changes, but the component is the same, Vue reuses the component (it is not rebuilt or remounted)
  - Example: switching from "/users/3" to "/users/8"
  - Component's **onMounted** hook is not executed, so we cannot use that for loading data that depends on the param (e.g., user with a specific id)
  - Instead, load data by watching the param (or mapped prop), or use onBeforeRouteUpdate navigation guard

```
watch(
  () => route.params.id,
  async (newId, oldId) => {
    // react to route changes
    await fetchUser(newId)
  }
)
```

```
onBeforeRouteUpdate(async (to, from) => {
  // react to route changes
  await fetchUser(to.params.id)
})
```

State & Pinia

State Management with Pinia

- Pinia (https://pinia.vuejs.org/) is Vue's official state management library
- Pinia allows us to create one or more stores
- A store centralizes reactive, shared application state
- Stores expose state (data), getters (derived, cached), and actions (side effects)
  - **State**: reactive data shared across components – in sync with all components' templates
  - **Getters**: derived, cached views of state. Like computed functions but applied to the store (filtered versions of the store's state).
  - **Actions**: functions (synchronous or asynchronous) that mutate state and handle side effects (e.g., business logic like loading or saving data on the server)
- Why use a store? share data across routes; avoid prop drilling; single source of truth
- Not for: purely local UI or easily derived single-component data
- Examples: authenticated user/session · local shopping cart · reference lists (countries, categories, etc.)

Store Definition

- Example of code that defines a Pinia store:

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import axios from 'axios'

export const useAuthStore = defineStore('auth', () => {
    const user = ref(null)

    const userName = computed(() => user.value?.name ?? '')

    async function fetchCurrentUser() {
        // side effect:
        const { data } = await axios.get('/api/me')
        // mutates state:
        user.value = data
    }

    return { user, userName, fetchCurrentUser }
})
```

state

getter

action

Defines what the store exposes to the components

Using the Store

- Example of code in a component using the previous Pinia store:

returns the store

```
<script setup>
import { onMounted } from 'vue'
import { useAuthStore } from '@/stores/auth'

const auth = useAuthStore()
onMounted(async () => {
   if (!auth.user)
      await auth.fetchCurrentUser()
})
</script>
<template>
  <p v-if="auth.userName">
    Welcome, {{ auth.userName }}</p>
  <p v-else>
    Welcome, guest</p>
</template>
```

state

action

getter

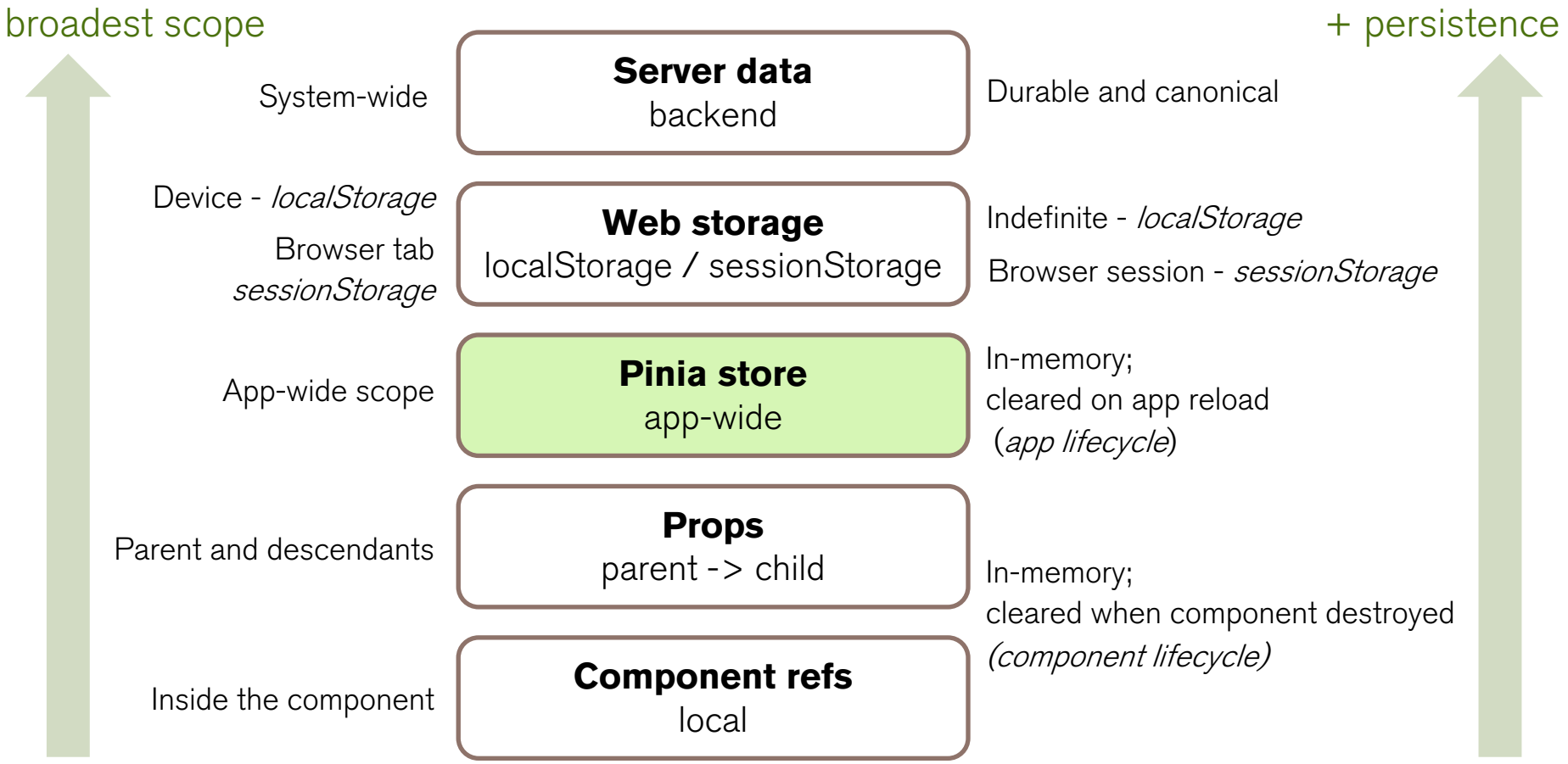
auth.user
does not require **.value**

Pinia unwraps refs on the store object

```
store:  auth
state:  auth.user
getter: auth.userName
action: auth.fetchCurrentUser()
```

State at Different Levels

- State exists at multiple levels, each with distinct scope and lifetime



Composables

Composable

- A composable is a function that uses Vue's Composition API to wrap reusable, stateful logic.
- It returns reactive values and helper functions that components can use.
- Call it inside `setup()` or the `<script setup>` section (top level)
- Each call creates its own isolated state.
- Useful for encapsulating reusable, cross-component behavior.
- VueUse (https://vueuse.org/) provides a collection of Vue Composition utilities

Example of a composable

- `useFetchJson`: minimal JSON fetch composable example:

```
import { ref } from 'vue'
export function useFetchJson(url, baseInit = {}) {
   const data = ref(null)
   const loading = ref(false)
   const error = ref(null)
   async function refresh(init = {}) {
      loading.value = true
      error.value = null
      try {
         const res = await fetch(url, { ...baseInit, ...init })
         if (!res.ok) throw new Error(`${res.status} ${res.statusText}`)
            data.value = await res.json()
      } catch (e) {
         error.value = e
      } finally {
         loading.value = false
      }
   }
   return { data, loading, error, refresh }
}
```

Return reactive values (`data`, `loading`, `error`) and the function `refresh()`

Using a composable

- Using the JSON fetch composable in a component:

```
<script setup>
  import { onMounted } from 'vue'
  import { useFetchJson } from '@/composables/useFetchJson'

  const { data, loading, error, refresh } = useFetchJson('/api/users')

  onMounted(refresh)
</script>

<template>
  <button @click="refresh" :disabled="loading">
    {{ loading ? 'Loading…' : 'Refresh' }}
  </button>
  <p v-if="error">Failed: {{ error.message }}</p>
  <pre v-else-if="data">{{ data }}</pre>
  <p v-else>Click refresh to load.</p>
</template>
```

Imports reactive values (`data, loading, error`) and the function `refresh()`

- Note: values returned by a composable are used in a component like local state and methods

References

Bibliography and Acknowledge

- Vue.js, https://vuejs.org/
- Vue.js, “Quick Start (Vue 3)” https://vuejs.org/guide/quick-start
- Vue.js, “Single-File Components (SFC)” https://vuejs.org/guide/scaling-up/sfc
- Vue.js, “Template Syntax” https://vuejs.org/guide/essentials/template-syntax
- Vue.js, “Class and Style Bindings” https://vuejs.org/guide/essentials/class-and-style
- Vue.js, “Forms — v-model” https://vuejs.org/guide/essentials/forms
- Vue.js, “Reactivity Fundamentals” https://vuejs.org/guide/essentials/reactivity-fundamentals
- Vue.js, “Computed Properties” https://vuejs.org/guide/essentials/computed
- Vue.js, “Watchers” https://vuejs.org/guide/essentials/watchers
- Vue.js, “Component Basics” https://vuejs.org/guide/essentials/component-basics
- Vue.js, “Props” https://vuejs.org/guide/components/props
- Vue.js, “Emits” https://vuejs.org/guide/components/events

- Vue.js, “Component v-model” https://vuejs.org/guide/components/v-model
- Vue.js, “Lifecycle Hooks” https://vuejs.org/guide/essentials/lifecycle
- Vue.js, “Provide / Inject” https://vuejs.org/guide/components/provide-inject
- Vue.js, “Slots” https://vuejs.org/guide/components/slots
- Vue.js, “Built-ins — KeepAlive, Teleport, Transition” https://vuejs.org/guide/built-ins/keep-alive
- Vue.js, “Composables” https://vuejs.org/guide/reusability/composables.html
- Vite, “Guide — Getting Started” https://vite.dev/guide
- Vue Router https://router.vuejs.org
- Pinia, “Getting Started” https://pinia.vuejs.org
- Vue Devtools, “Guide — Vue Devtools” https://devtools.vuejs.org/guide
- VueUse, “Collection of Vue Composition Utilities” https://vueuse.org
- MDN Web Docs, “Fetch API” https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Acknowledge

The authors acknowledge the use of ChatGPT (https://chatgpt.com/) and Google Gemini (https://gemini.google.com) as a tool supporting content development and text composition. Full responsibility for the final content rests with the authors.